User API Contracts

Poker Application — Frontend Integration Reference

These contracts are locked. Request/response shapes must not change without updating the user frontend.

Auth

POST /api/auth/otp/request

File `ApiCaller.js → requeastOtp_Post`
Auth `None`

REQUEST BODY

```
{ mobileNumber: string }
```

RESPONSE

```
{ message: string }
```

Note: Rate limited: 3 requests per 10 minutes then blocked for 10 minutes.

/api/auth/otp/verify

File `ApiCaller.js → verifyLogin_Post`
Auth `None`

REQUEST BODY

```
{
  mobileNumber: string,
  otp: string,
  deviceType?: "android" | "ios" | "unknown"
}
```

RESPONSE

```
{
  message: string,
  token: string,
  userName: string,
  userId: string,
  wallet: {
    balance: number,
    instantBonus: number,
    lockedBonus: number
  }
}
```

Note: Creates user and wallet on first login. Issues JWT Bearer token. Deletes OTP record after successful verification.

Lobby

GET **/api/lobby/games**

File `ApiCaller.js → getAllPoker_Get`
Auth `Bearer token (Authorization header)`

RESPONSE

```
[
  {
    pokerId: string,
    gameType: "Texas Hold'em" | "Omaha" | "Seven-Card Stud" | "Razz" | "Five-Card
Draw",
    objective: string,
    description: string,
    status: "active" | "maintenance",
    pokerModes: [
      {
        pokerModeId: string,
        mode: "cash" | "practice",
        minBuyIn: number,
        maxBuyIn: number,
        bType: "blinds" | "antes",
        smallBlind?: number,    // present if bType === "blinds"
        bigBlind?: number,      // present if bType === "blinds"
        anteAmount?: number,    // present if bType === "antes"
        totalSeats: number,
        livePlayers: number
      }
    ]
  }
]
```

Note: Returns all active and maintenance poker games with live desk stats per mode.

GET **/api/lobby/desks/best?pokerModeId=**

File `ApiCaller.js → findPokerDesk_Post (now GET)`
Auth `Bearer token (Authorization header)`

REQUEST BODY

```
Query param: pokerModeId (string, required)
```

RESPONSE

```
{
  id: string,
  tableName: string,
  maxSeats: number,
  minBuyIn: number,
  maxBuyIn: number,
  pokerModeId: string,
  seats: [
    {
      userId: string,
      username: string,
      seatNumber: number,
      buyInAmount: number,
      balanceAtTable: number,
      status: "active" | "disconnected" | "sittingOut"
    }
  ],
  message: string
}
```

Note: Changed from POST to GET. Frontend ApiCaller.js must be updated to send pokerModeId as query param instead of request body.

Wallet

GET /api/user/wallet

File `ApiCaller.js → fetchUserWallet_Get`
Auth `Bearer token (Authorization header)`

RESPONSE

```
{
  message: string,
  wallet: {
    balance: number,
    instantBonus: number,
    lockedBonus: number
  }
}
```

Note: Migrated from /api/auth/fetchUserWallet. Same response shape.

GET **/api/user/wallet/transactions**

File	`ApiCaller.js → getWalletTransaction_Get`
Auth	`Bearer token (Authorization header)`

REQUEST BODY

```
Query params (all optional):
  page: number (default 1)
  limit: number (default 10, max 50)
  type: "deposit" | "withdraw" | "deskIn" | "deskWithdraw" | "bonus" | "pgDeposit"
  status: "pending" | "completed" | "failed" | "reversed"
  startDate: string (ISO date)
  endDate: string (ISO date)
```

RESPONSE

```
{
  message: string,
  transactions: WalletTransaction[],
  page: number,
  limit: number,
  totalTransactions: number,
  totalPages: number
}
```

Note: Migrated from /api/auth/getWalletTransaction. Now queries separate WalletTransaction collection.

Banks

GET **/api/user/banks**

File `ApiCaller.js → getBankAcc_Get`
Auth `Bearer token (Authorization header)`

REQUEST BODY

```
Query params (optional):
  page: number (default 1)
  limit: number (default 10)
```

RESPONSE

```
{
  bankAccounts: BankAccount[],
  totalPages: number,
  currentPage: number,
  totalBankAccounts: number
}
```

Note: Migrated from /api/auth/getBankAcc. Max 5 bank accounts per user enforced at model level.

/api/user/banks

File ApiCaller.js → addBankAcc_Post
Auth Bearer token (Authorization header)

REQUEST BODY

```
{
  accountNumber: string,
  bankName: string,
  ifscCode: string,
  accountHolderName: string,
  isDefault?: boolean (default false)
}
```

RESPONSE

```
{
  message: string,
  bankAccount: BankAccount
}
```

Note: Migrated from /api/auth/addBankAcc. Setting isDefault=true automatically unsets all other default accounts.

GET /api/user/banks/transactions

File `ApiCaller.js → getBankTransactions_Get`
Auth `Bearer token (Authorization header)`

REQUEST BODY

```
Query params (optional):
  page: number (default 1)
  limit: number (default 10, max 50)
  type: "deposit" | "withdraw"
  status: "pending" | "completed" | "failed"
  startDate: string (ISO date)
  endDate: string (ISO date)
```

RESPONSE

```
{
  message: string,
  transactions: BankTransaction[],   // populated with bankAccount details
  page: number,
  limit: number,
  totalTransactions: number,
  totalPages: number
}
```

Note: Migrated from /api/auth/getBankTransactions. bankAccountId field is populated with account details.

POST

/api/user/banks/transactions

File	`ApiCaller.js → createBankTransaction_Post`
Auth	`Bearer token (Authorization header)`

REQUEST BODY

```
{
  bankAccountId: string,
  amount: number,
  type: "deposit" | "withdraw",
  remark?: string,
  imageUrl?: string   // required when type === "deposit"
}
```

RESPONSE

```
{
  message: string,
  transaction: BankTransaction
}
```

Note: Migrated from /api/auth/createBankTransaction. imageUrl is required for deposits, ignored for withdrawals. Withdrawal checks wallet balance before creating.

Payments (Razorpay)

/api/payments/razorpay/order

File `ApiCaller.js → createPayment_Post`
Auth `Bearer token (Authorization header)`

REQUEST BODY

```
{
  amount: number,
  currency?: string  // default "INR"
}
```

RESPONSE

```
{
  message: string,
  order: RazorpayOrder,
  transactionId: string
}
```

Note: Migrated from /api/auth/payment/createPayment. Creates a GatewayTransaction record before calling Razorpay. Amount is in rupees (converted to paise internally).

POST

/api/payments/razorpay/verify

File	`ApiCaller.js → verifyPayment_Post`
Auth	`Bearer token (Authorization header)`

REQUEST BODY

```
{
  razorpay_payment_id: string,
  razorpay_order_id: string,
  razorpay_signature: string
}
```

RESPONSE

```
{
  message: string
}
```

Note: Migrated from /api/auth/payment/verifyPayment. Verifies HMAC signature, credits wallet balance, creates WalletTransaction record.

Game History (New)

GET **/api/user/games/history**

File ApiCaller.js → (to be added)
Auth Bearer token (Authorization header)

REQUEST BODY

```
Query params (optional):
  page: number (default 1)
  limit: number (default 10, max 50)
  gameType: "Texas Hold'em" | "Omaha" | "Seven-Card Stud" | "Razz" | "Five-Card Dr
aw"
```

RESPONSE

```
{
  message: string,
  games: [
    {
      gameId: string,
      gameType: string,
      tableName: string,
      startedAt: Date,
      completedAt: Date,
      totalPot: number,
      myResult: {
        startingStack: number,
        endingStack: number,
        totalBet: number,
        isWinner: boolean
      }
    }
  ],
  page: number,
  limit: number,
  totalGames: number,
  totalPages: number
}
```

Note: New endpoint not in original migration list. Must be added to ApiCaller.js on the frontend. Queries PokerGameArchive collection.

All user-facing endpoints use Bearer token authentication via the Authorization header.
Token is issued by /api/auth/otp/verify and expires based on JWT_EXPIRES_IN environment variable.